



Sistema de Gestión de Órdenes de Servicio para Taller Mecánico

Estudiante: Francisco R. Diaz

Matrícula: 2023-3300016

Asignatura: Ingeniería de Software

Profesor/a: Ing. Carlos Artemio Escalante Lorenzo

Enlace del video: https://youtu.be/0lozuXjt_9M

Fecha: 25/06/2026

TABLA DE CONTENIDO

Contenido

1. Identificación y Descripción del Sistema Desarrollado.....	3
1.1 Usuarios Objetivo	3
1.2 Alcance Funcional	3
1.3 Objetivos de Ingeniería	3
1.4 Stack Tecnológico.....	4
2. Fundamentos de Diseño de Software Aplicados al Sistema	4
2.5 Diagrama UML de Paquetes / Componentes	5
3. Arquitectura del Sistema	7
4. Patrones de Diseño Implementados	10
4.2 Patrón State (Comportamiento).....	10
4.3 Patrón Strategy (Comportamiento).....	12
4.4 Patrón Builder (Creacional).....	14
4.5 Patrón Observer (Comportamiento) — EventBus	15
4.6 Patrón Facade (Estructural) — TallerFacade.....	16
5. Calidad del Software y Proceso de Desarrollo	17
6. Reflexión Crítica y Mejoras Propuestas	19
6.2 Plan de Mejora para la Siguiente Iteración.....	19
6.3 Conclusión de la Reflexión Crítica	20
Referencias Bibliográficas.....	22

1. Identificación y Descripción del Sistema Desarrollado

El presente sistema desarrollado durante el 2do cuatrimestre 2026 «Sistema de Gestión de Órdenes de Servicio para Taller Mecánico» (SGOSTM). El propósito central es digitalizar y automatizar el ciclo de vida completo de los servicios mecánicos: desde la recepción inicial de un vehículo hasta la facturación definitiva al cliente, pasando por diagnóstico, presupuestación, aprobación del cliente, ejecución de la reparación, control de calidad y entrega. El sistema reemplaza los procesos manuales en papel o en hojas de cálculo que caracterizan a la mayoría de los talleres mecánicos medianos de la República Dominicana.

1.1 Usuarios Objetivo

Rol	Responsabilidades en el sistema
Administrador	Acceso irrestricto; gestión de usuarios, reportes y configuración global.
Recepcionista	Creación de órdenes, registro de clientes y vehículos.
Mecánico	Visualización de sus órdenes asignadas; actualización de progreso.
Cajero	Aprobación de presupuestos y generación de facturas.

1.2 Alcance Funcional

- Gestión de clientes (registro, historial, marcado de cliente frecuente).
- Gestión de vehículos (marca, modelo, año, placa, kilometraje, historial de servicios).
- Órdenes de servicio con ciclo de vida completo de ocho estados.
- Presupuestación automática con cálculo de ITBIS (18 %) y estrategias de descuento.
- Asignación y seguimiento de mecánicos con dashboard de carga de trabajo.
- Facturación con exportación en PDF.
- Dashboard de reportes: ingresos, estados de órdenes y productividad de mecánicos.
- Autenticación JWT con cuatro roles diferenciados.
- Notificaciones por correo electrónico (Nodemailer) y SMS (Twilio).

1.3 Objetivos de Ingeniería

Los objetivos de ingeniería que guiaron el desarrollo del SGOSTM fueron: (a) aplicar principios de Diseño Orientado a Objetos (OOP) y principios SOLID en cada capa del sistema; (b) implementar una arquitectura limpia que aisle la lógica de negocio de los detalles tecnológicos; (c) aplicar al menos seis patrones de diseño Gang of Four (GoF) en contextos reales y justificados; (d) garantizar la mantenibilidad y extensibilidad del código mediante cohesión alta y acoplamiento bajo; y (e) proveer una API REST documentada para el consumo desde el frontend React.

1.4 Stack Tecnológico

Capa	Tecnología / Herramienta
Frontend	React 18 + Vite + Tailwind CSS
Backend	Node.js 20 + Express 4
Base de datos	PostgreSQL 16
ORM	Prisma 5
Autenticación	JSON Web Tokens (jsonwebtoken + bcryptjs)
Notificaciones	Nodemailer (email) / Twilio (SMS)
Testing	Jest + Supertest
Contenedores	Docker + Docker Compose
Control versiones	Git + GitHub
Despliegue	Render (backend Web Service + frontend Static Site)

2. Fundamentos de Diseño de Software Aplicados al Sistema

Los cuatro principios fundamentales del diseño de software abstracción, modularidad, cohesión y acoplamiento no fueron aplicados como conceptos abstractos aislados, sino como guías de decisión concretas que determinaron cada componente del SGOSTM. A continuación, estaré explicando cómo cada principio se manifiesta en la arquitectura del sistema (Gamma et al., 1994; Martin, 2017).

2.1 Abstracción

La abstracción se materializa en tres niveles. En primer lugar, la clase abstracta EstadoOrden define el contrato que todos los estados concretos deben cumplir (métodos avanzar, rechazar, nombre y esTerminal), sin exponer detalles de transición. En segundo lugar, los Value Objects OrdenId, Prioridad y Dinero encapsulan conceptos del dominio con validación interna, resultando en tipos semánticamente ricos. En tercer lugar, los repositorios se definen como interfaces en la capa de dominio (por ejemplo, IOrdenRepository), ocultando completamente el detalle de acceso a PostgreSQL vía Prisma.

2.2 Modularidad

El sistema se organiza en cuatro módulos independientes alineados con la Arquitectura Hexagonal: domain, application, infrastructure e interfaces. Cada módulo tiene responsabilidades bien delimitadas y puede evolucionar de forma independiente. Por ejemplo, reemplazar PostgreSQL por MongoDB implicaría únicamente cambios en el módulo infrastructure, sin tocar dominio ni aplicación. La herramienta dependency-cruiser valida en tiempo de integración continua que ningún módulo de dominio importe de infraestructura.

2.3 Cohesión

Cada clase del sistema posee una única razón de cambio (Principio de Responsabilidad Única). OrdenServicio gestiona exclusivamente el ciclo de vida de la orden. CalculadoraOrden se ocupa únicamente del cálculo de totales e impuestos. TallerFacade actúa como punto de entrada unificado delegando cada operación al caso de uso correspondiente. EventBus gestiona únicamente la publicación y suscripción de Domain Events. Esta alta cohesión facilita el mantenimiento y la prueba unitaria de cada componente.

2.4 Acoplamiento

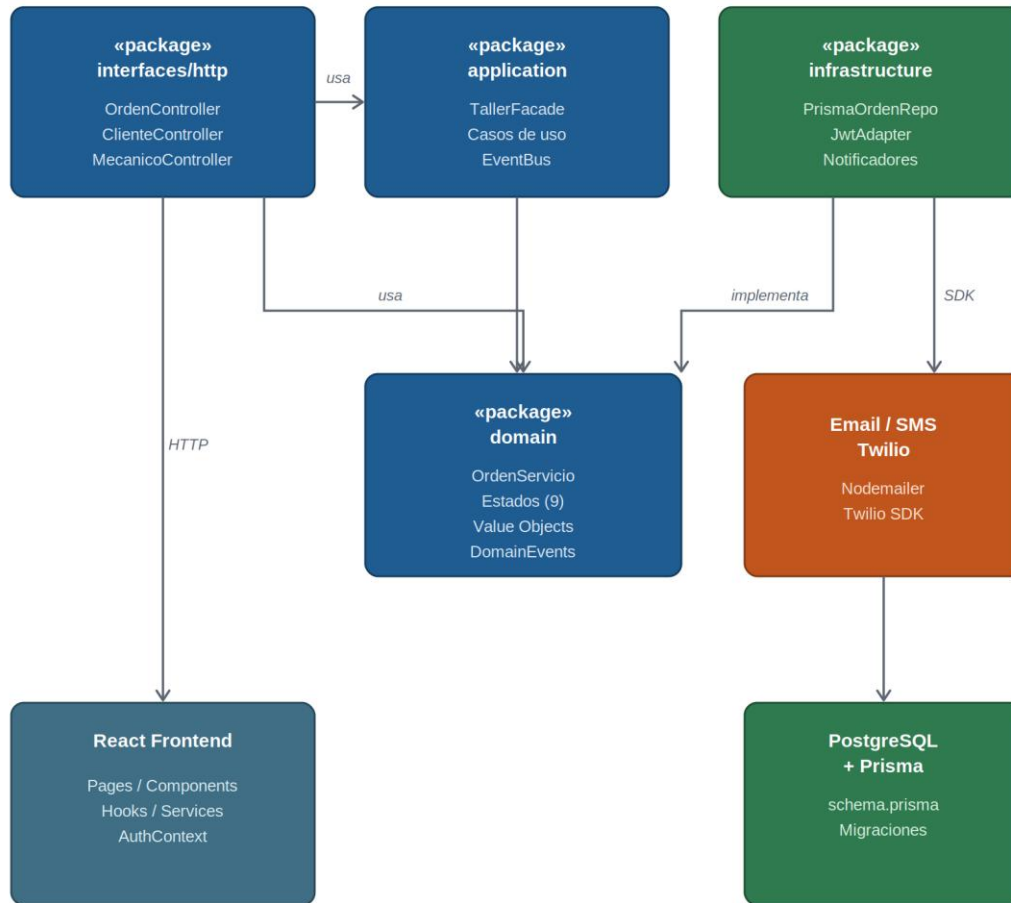
El acoplamiento se redujo al mínimo mediante el Principio de Inversión de Dependencias (DIP). Los controladores HTTP no conocen los repositorios concretos: solo interactúan con TallerFacade. Los casos de uso reciben sus dependencias (repositorios, EventBus) por inyección de dependencias a través de container.js. Los estados concretos no se acoplan entre sí: cada uno solo conoce el estado al que puede transicionar. Esta estructura permite sustituir cualquier componente de infraestructura sin modificar la lógica de negocio (Cockburn, 2005; Evans, 2003).

2.5 Diagrama UML de Paquetes / Componentes

El siguiente diagrama de paquetes muestra las dependencias entre las cuatro capas del sistema. Las flechas representan dependencias en dirección inwards (hacia el dominio), respetando la Regla de Dependencia de la Arquitectura Limpia (Martin, 2017):

Capa	Contiene	Depende de
«interfaces» http/controllers	OrdenController ClienteController MecanicoController	application (TallerFacade)
«application»	TallerFacade Casos de uso EventBus	domain (interfaces y entidades)
«infrastructure»	PrismaOrdenRepo JwtAdapter Notificadores	domain (interfaces)
«domain»	OrdenServicio Estados Value Objects Domain Events	Ninguna capa externa

Figura 8. Diagrama de Paquetes y Componentes — SGOSTM



2.6 Principios SOLID

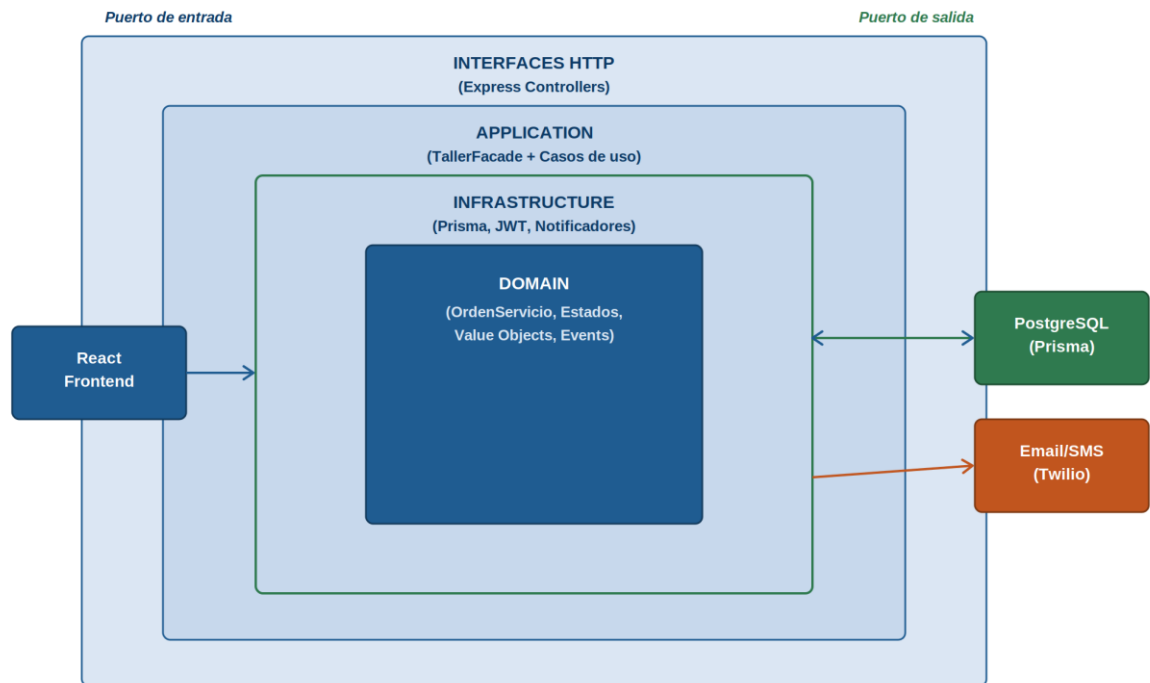
Principio	Aplicación en SGOSTM
SRP — Responsabilidad Única	Cada clase tiene una sola razón de cambio (OrdenServicio, CalculadoraOrden, EventBus).
OCP — Abierto/Cerrado	Nuevos estados y estrategias de descuento se agregan sin modificar clases existentes.
LSP — Sustitución de Liskov	Todos los EstadoConcreto son intercambiables con EstadoOrden sin alterar el comportamiento.
ISP — Segregación de I.	Los controladores solo usan los métodos de TallerFacade que necesitan.
DIP — Inversión de Dep.	Los casos de uso dependen de interfaces de repositorio, no de implementaciones Prisma.

3. Arquitectura del Sistema

3.1 Arquitectura Seleccionada: Hexagonal (Ports & Adapters) con DDD Táctico

La arquitectura seleccionada para el SGOSTM es la Arquitectura Hexagonal, propuesta originalmente por Cockburn (2005) y ampliamente adoptada en sistemas que implementan Domain-Driven Design. Esta arquitectura establece que el núcleo de negocio (dominio) debe ser independiente de cualquier tecnología externa, comunicándose con el exterior únicamente a través de Puertos (interfaces abstractas) y Adaptadores (implementaciones concretas).

Figura 1. Diagrama de Arquitectura Hexagonal — SGOSTM



3.2 Justificación de la Selección Arquitectónica

Requisito de independencia tecnológica: El dominio no debe conocer si la persistencia es PostgreSQL, MongoDB o un repositorio en memoria para tests.

Requisito de testeabilidad: Al inyectar repositorios en memoria, los casos de uso se prueban sin base de datos real.

Requisito de extensibilidad: Agregar un nuevo canal de notificación (WhatsApp, push) solo requiere un nuevo Adaptador.

Requisito académico: La arquitectura permite demostrar DDD táctico con Aggregate, Value Objects y Domain Events sin fricción tecnológica.

3.3 Capas y Responsabilidades

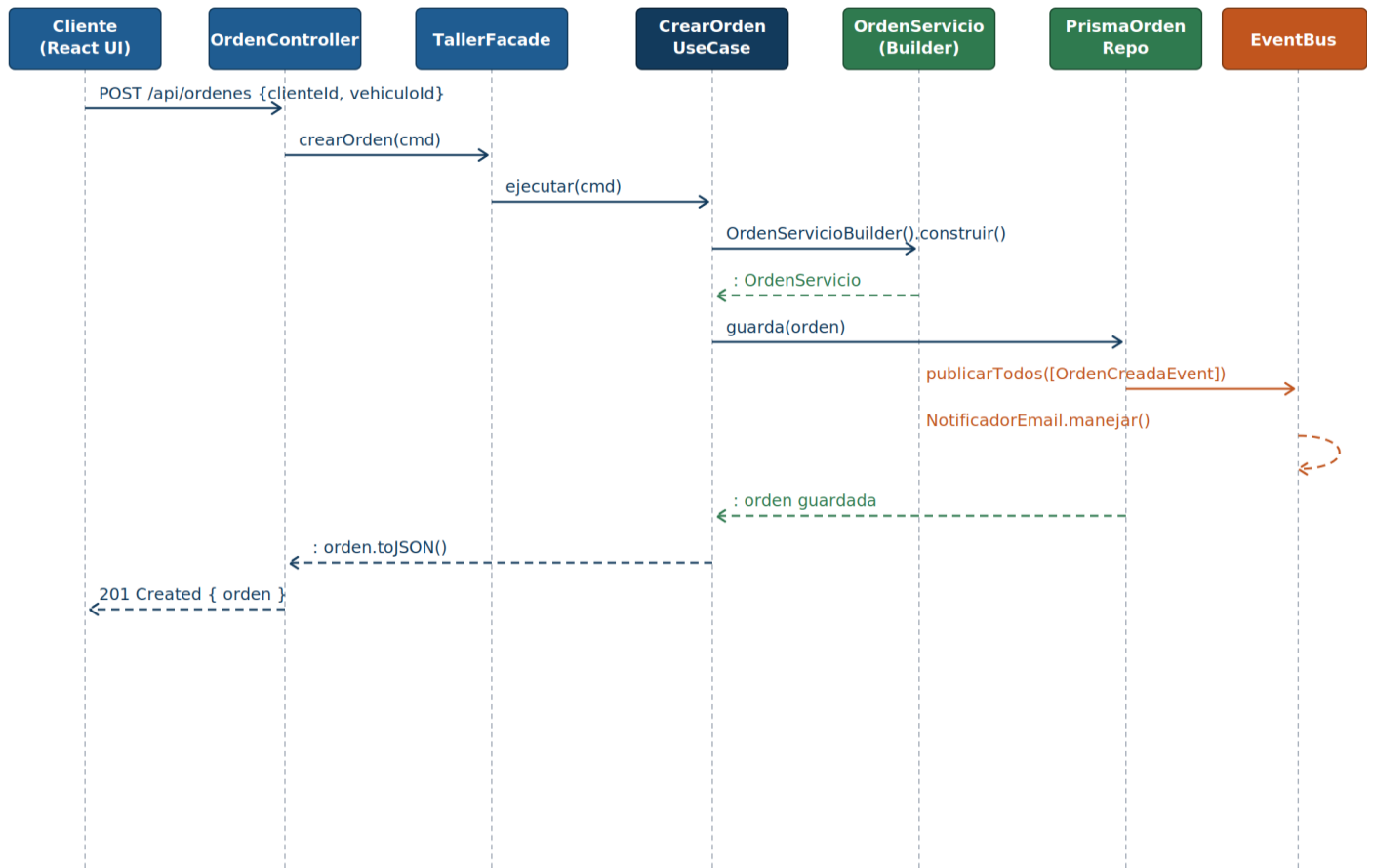
Capa	Carpeta	Responsabilidad principal
Domain	backend/src/domain/	Lógica de negocio pura. Aggregate, Value Objects, Estados, Domain Events, interfaces de repositorios.
Application	backend/src/application/	Casos de uso (orquestación). TallerFacade, EventBus. Sin lógica de negocio propia.
Infrastructure	backend/src/infrastructure/	Implementaciones concretas: PrismaOrdenRepo, JwtAdapter, NotificadorEmail, NotificadorSMS.
Interfaces HTTP	backend/src/interfaces/	Controladores Express, validación de entrada, serialización de respuestas JSON.
Frontend React	frontend/src/	Páginas, componentes, hooks y servicios HTTP para consumo del API REST.

3.4 Diagrama Arquitectónico Completo

El siguiente esquema describe las interacciones entre componentes del sistema de extremo a extremo:

Componente origen	→	Componente destino	Mecanismo
Navegador (React)	→	OrdenController (Express)	HTTP REST (JSON)
OrdenController	→	TallerFacade	Llamada de método (DI)
TallerFacade	→	CrearOrdenUseCase	Delegación al caso de uso
CrearOrdenUseCase	→	OrdenServicio (Aggregate)	Factory method .crear()
CrearOrdenUseCase	→	IOrdenRepository.guardar()	Puerto de salida (interfaz)
PrismaOrdenRepo	→	PostgreSQL	Prisma ORM (adaptador)
PrismaOrdenRepo (guardar)	→	EventBus.publicarTodos()	Domain Events (pull)
EventBus	→	NotificadorEmail / SMS	Observadores suscritos

Figura 7. Diagrama de Secuencia — Crear Orden de Servicio



3.5 Decisiones de Arquitectura Adicionales

- CommonJS (require/module.exports) sobre ESM: mejor compatibilidad con Jest sin configuración adicional.
- JWT simple sobre OAuth2/Keycloak: suficiente para el alcance académico; migratable en producción.
- Inyección de dependencias manual (container.js): evita librerías adicionales manteniendo el principio DIP.
- Patrón pull para Domain Events: los eventos se acumulan en el Aggregate y se publican al guardar.
- dependency-cruiser para validación arquitectónica: equivalente a ArchUnit para ecosistema Node.js.

4. Patrones de Diseño Implementados

El SGOSTM implementa siete patrones de diseño del catálogo GoF (Gamma et al., 1994), distribuidos en las tres categorías canónicas, lo cual se analizan en profundidad los tres más representativos del sistema, incluyendo el problema que resuelven, su implementación concreta y fragmentos de código real del sistema.

4.1 Resumen de Patrones Implementados

Categoría	Patrón	Clase / Archivo principal
Creacional	Builder	OrdenServicioBuilder.js
Creacional	Factory Method	ServicioFactory.js
Estructural	Facade	TallerFacade.js
Comportamiento	State	EstadoOrden.js + estados concretos
Comportamiento	Observer	EventBus.js + handlers de notificación
Comportamiento	Strategy	CalculadoraOrden.js + estrategias de descuento
Comportamiento	Repository	IOrdenRepository (interfaz) + PrismaOrdenRepo

4.2 Patrón State (Comportamiento)

Problema que resuelve

Una OrdenServicio atraviesa ocho estados distintos (RECIBIDA, EN_DIAGNOSTICO, PRESUPUESTADA, APROBADA, EN_REPARACION, LISTA, ENTREGADA, FACTURADA) más un estado de rechazo. Sin el patrón State, la lógica de transición requeriría una cadena de condicionales (if/else o switch) en OrdenServicio, violando el principio OCP y generando un código frágil y difícil de mantener (Gamma et al., 1994).

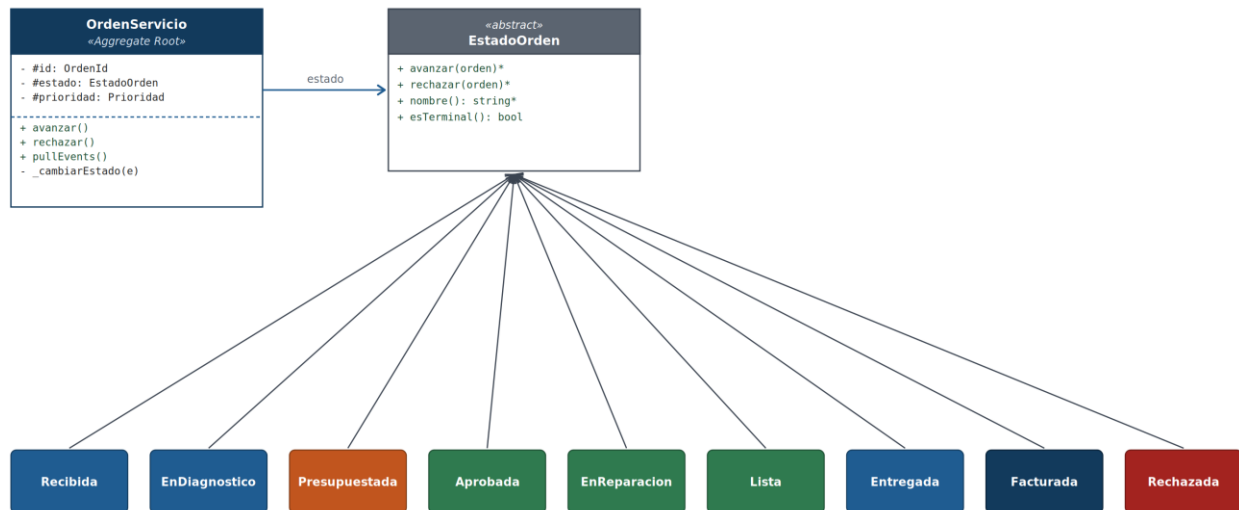
Implementación en el sistema

La clase abstracta EstadoOrden define el contrato con los métodos avanzar(ordén), rechazar(ordén), nombre() y esTerminal(). Cada estado concreto (EstadoRecibida, EstadoEnDiagnostico, etc.) hereda de esta clase e implementa únicamente las transiciones permitidas desde ese estado. El Aggregate OrdenServicio mantiene una referencia al estado actual y delega las operaciones de transición al objeto de estado. Esto cumple el principio OCP: agregar un nuevo estado no modifica OrdenServicio ni los estados existentes.

Figura 2. Diagrama de Máquina de Estados – Ciclo de Vida de OrdenServicio



Figura 3. Diagrama de Clases – Patrón State



Código representativo

```

// EstadoOrden.js – Clase base abstracta (contrato del State)
class EstadoOrden {
  avanzar(orden) { throw new Error('Acción no permitida en estado ' + this.nombre()); }
  rechazar(orden) { throw new Error('Acción no permitida en estado ' + this.nombre()); }
  nombre() { throw new Error('nombre() debe ser implementado'); }
  esTerminal() { return false; }
}
  
```

```
// EstadoRecibida.js - Estado concreto inicial
class EstadoRecibida extends EstadoOrden {
  avanzar(orden) { orden._cambiarEstado(new EstadoEnDiagnostico()); }
  nombre()       { return 'RECIBIDA'; }
}

// EstadoPresupuestada.js - único estado que permite rechazar()
class EstadoPresupuestada extends EstadoOrden {
  avanzar(orden) { orden._cambiarEstado(new EstadoAprobada()); }
  rechazar(orden) { orden._cambiarEstado(new EstadoRechazada()); }
  nombre()       { return 'PRESUPUESTADA'; }
}

// OrdenServicio.js - usa el estado actual
avanzar() {
  if (this.#estado.esTerminal()) throw new Error('Orden en estado terminal');
  this.#estado.avanzar(this); // delega al estado actual
}
```

4.3 Patrón Strategy (Comportamiento)

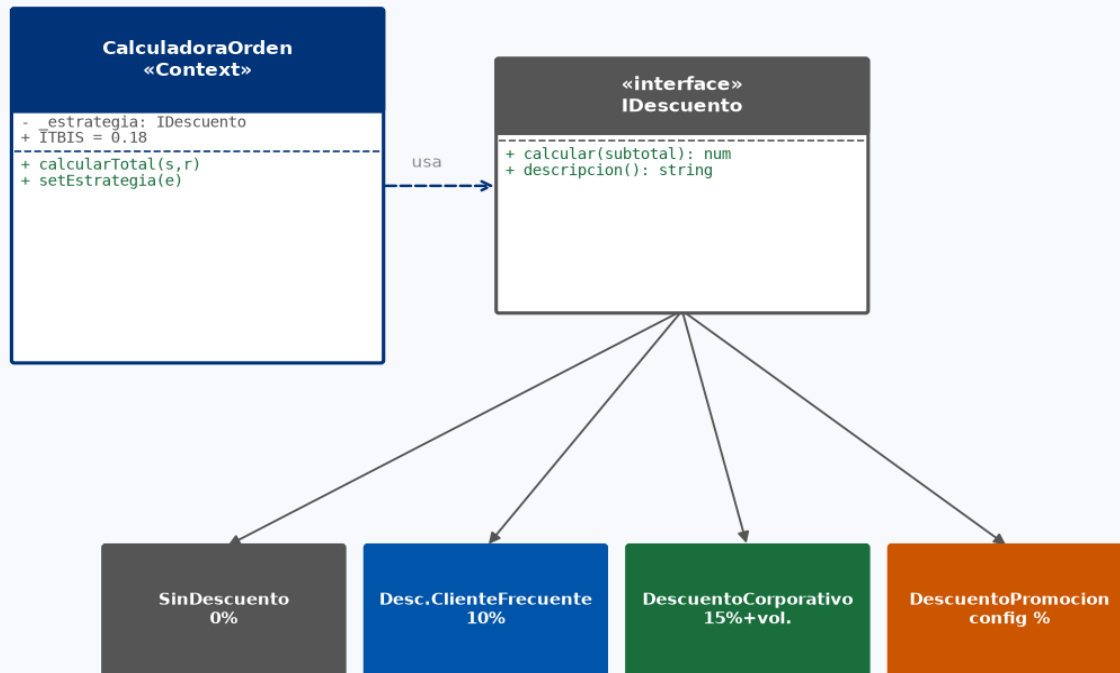
Problema que resuelve

El cálculo del precio final de una orden varía según el perfil del cliente: sin descuento (cliente nuevo), descuento por cliente frecuente (10 %), descuento corporativo (15 % fijo más descuento por volumen), o descuento por promoción (porcentaje configurable). Sin el patrón Strategy, la clase CalculadoraOrden acumularía condicionales que violarían OCP y SRP. Además, la normativa dominicana exige aplicar ITBIS (18 %) sobre la base imponible después del descuento (DGII, 2024).

Implementación en el sistema

CalculadoraOrden actúa como Contexto del patrón. Recibe por inyección una estrategia concreta que implementa los métodos calcular(subtotal) y descripcion(). El método setEstrategia() permite cambiar el algoritmo de descuento en tiempo de ejecución. El GenerarPresupuestoUseCase selecciona la estrategia correcta según los atributos del cliente y la configura en la calculadora antes de invocar calcularTotal().

Figura 4. Diagrama de Clases — Patrón Strategy (Descuentos + ITBIS)



Código representativo

```

// CalculadoraOrden.js - Contexto Strategy
class CalculadoraOrden {
  static ITBIS = 0.18;
  constructor(estrategia = new SinDescuento()) { this._estrategia = estrategia; }
  setEstrategia(estrategia) { this._estrategia = estrategia; }

  calcularTotal(servicios, repuestos) {
    const subtotal = servicios.reduce((a,s) => a + s.calcularTotal(), 0)
    + repuestos.reduce((a,r) => a + r.precio * r.cantidad, 0);
    const descuento = this._estrategia.calcular(subtotal);
    const baseImponible = subtotal - descuento;
    const itbis = parseFloat((baseImponible *
CalculadoraOrden.ITBIS).toFixed(2));
    const total = parseFloat((baseImponible + itbis).toFixed(2));
    return { subtotal, descuento,
      descuentoDescripcion: this._estrategia.descripcion(),
      baseImponible, itbis, total };
  }
}

// DescuentoClienteFrecuente.js - Estrategia concreta
class DescuentoClienteFrecuente {
  calcular(subtotal) { return subtotal * 0.10; } // 10 %
  descripcion() { return 'Descuento cliente frecuente (10%)'; }
}
    
```

4.4 Patrón Builder (Creacional)

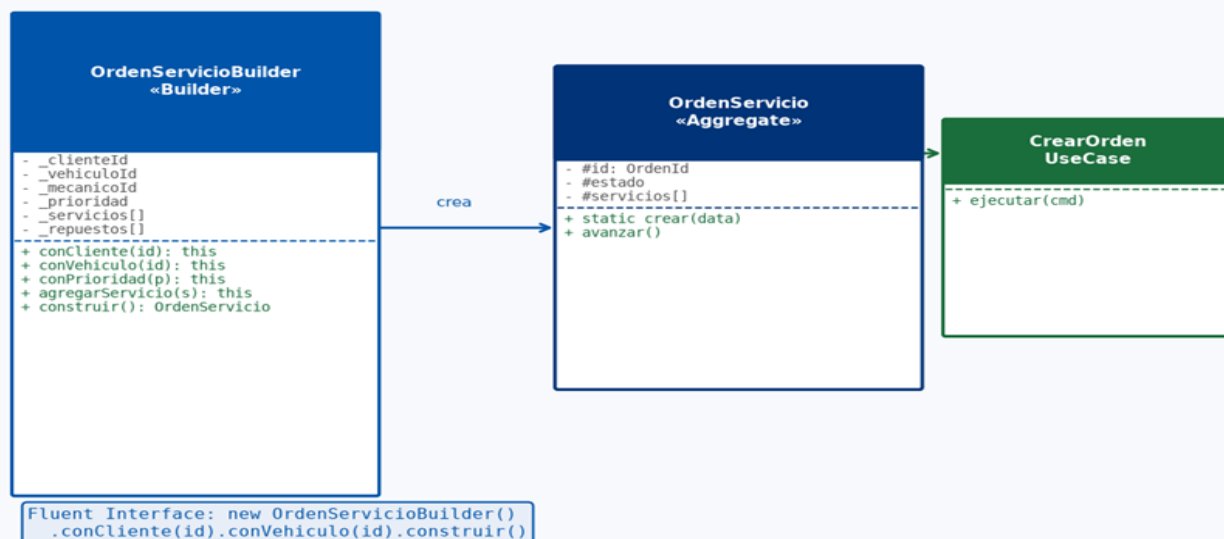
Problema que resuelve

Una `OrdenServicio` requiere múltiples campos para su construcción: `clienteId`, `vehiculoId`, `mecanicoId` (opcional), `prioridad`, `lista de servicios`, `repuestos`, `notas` y `fecha estimada`. Un constructor con ocho parámetros es ilegible, propenso a errores de orden y dificulta la validación previa. El patrón Builder resuelve este problema permitiendo construir el objeto paso a paso con una interfaz fluida (fluent interface), validando al final (Gamma et al., 1994).

Implementación en el sistema

`OrdenServicioBuilder` encapsula todos los parámetros de construcción. Cada método de configuración retorna `this` para habilitar el encadenamiento fluido. El método `construir()` valida los campos obligatorios (`clienteId`, `vehiculoId` y al menos un servicio) antes de invocar `OrdenServicio.crear()`. El `CrearOrdenUseCase` utiliza este builder para garantizar que toda orden creada sea estructuralmente válida.

Figura 5. Diagrama de Clases — Patrón Builder



Código representativo

```
// OrdenServicioBuilder.js
class OrdenServicioBuilder {
  constructor() {
    this._clienteId = null; this._vehiculoId = null;
    this._prioridad = 'NORMAL'; this._servicios = []; this._repuestos = [];
  }
  conCliente(id) { this._clienteId = id; return this; }
  conVehiculo(id) { this._vehiculoId = id; return this; }
```

```

conPrioridad(p)      { this._prioridad = p;    return this; }
agregarServicio(s)   { this._servicios.push(s); return this; }
agregarRepuesto(r)   { this._repuestos.push(r); return this; }
conNotas(n)          { this._notas = n;        return this; }

construir() {
  if (!this._clienteId)      throw new Error('clienteId requerido');
  if (!this._vehiculoId)     throw new Error('vehiculoId requerido');
  if (this._servicios.length===0) throw new Error('Al menos un servicio
requerido');
  return OrdenServicio.crear({ ...this });
}
}

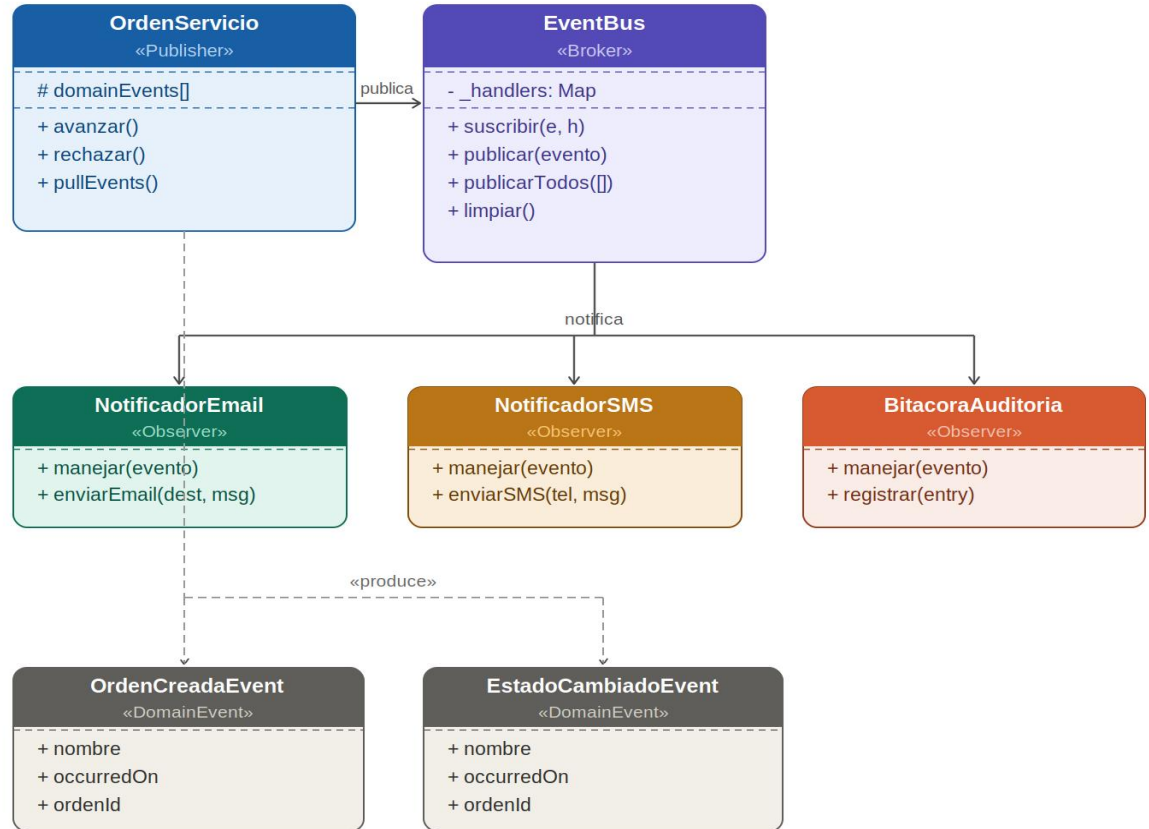
// Uso en CrearOrdenUseCase.js
const orden = new OrdenServicioBuilder()
  .conCliente(cmd.clienteId)
  .conVehiculo(cmd.vehiculoId)
  .conPrioridad(cmd.prioridad || 'NORMAL')
  .agregarServicio(servicioCambioAceite)
  .conNotas('Cliente reporta ruido al frenar')
  .construir();

```

4.5 Patrón Observer (Comportamiento) — EventBus

El patrón Observer se implementa a través del EventBus. Los Domain Events (OrdenCreadaEvent, EstadoCambiadoEvent) son generados por el Aggregate OrdenServicio y extraídos por el repositorio mediante el método pullEvents(). El repositorio invoca eventBus.publicarTodos() enviando cada evento a todos los handlers suscritos (NotificadorEmail, NotificadorSMS, BitacoraAuditoria). Esta arquitectura desacopla completamente el dominio de los canales de notificación.

Diagrama de Clases — Patrón Observer (EventBus + Domain Events)



4.6 Patrón Facade (Estructural) — TallerFacade

TallerFacade provee una interfaz unificada hacia el subsistema complejo formado por doce casos de uso independientes. Los controladores HTTP solo se comunican con TallerFacade, sin conocer la existencia de casos de uso individuales ni de repositorios. Esto implementa el principio ISP (los controladores usan solo lo que necesitan) y facilita el mantenimiento centralizado de la capa de aplicación.

5. Calidad del Software y Proceso de Desarrollo

5.1 Metodología de Desarrollo

El desarrollo del SGOSTM siguió una metodología iterativa e incremental con enfoque ágil adaptado al contexto individual del proyecto. Se organizó en seis sprints de aproximadamente cuatro días cada uno, con entregables funcionales al cierre de cada iteración. El backlog de tareas se gestionó mediante un tablero TODO.md versionado en Git. Las decisiones arquitectónicas relevantes se documentaron en AGENTS.md, incluyendo la justificación de cada decisión asistida por herramientas de IA.

Sprint	Entregable principal
Sprint 1	Scaffolding completo: estructura hexagonal, Prisma schema, Docker Compose.
Sprint 2	Domain layer: OrdenServicio, patrón State completo (9 estados), Value Objects.
Sprint 3	Application layer: casos de uso, TallerFacade, EventBus con handlers.
Sprint 4	Infraestructura: repositorios Prisma, JWT, Nodemailer, container.js.
Sprint 5	Interfaces HTTP: controladores, middleware de autenticación, rutas REST.
Sprint 6	Frontend React, integración completa, despliegue en Render.

5.2 Garantía de Calidad

- Pruebas unitarias con Jest para OrdenServicio, CalculadoraOrden y casos de uso críticos.
- Pruebas de integración con Supertest para endpoints REST del API.
- Validación arquitectónica con dependency-cruiser: el dominio no puede importar de infraestructura.
- Revisión manual del código contra los principios SOLID documentados en cada clase.
- Control de versiones con Git: commits atómicos por funcionalidad, ramas por sprint.
- Documentación inline con JSDoc en todas las clases y métodos públicos.
- Uso de variables de entorno para configuración sensible (.env.example provisto).

5.3 Métricas de Calidad ISO/IEC 25010

La norma ISO/IEC 25010 (2023) define un modelo de calidad del producto software organizado en ocho características principales. Para el SGOSTM se proponen y aplican las siguientes dos métricas:

Métrica 1: Mantenibilidad — Modularidad y Analizabilidad

Definición: Porcentaje de casos de uso con cobertura de prueba unitaria $\geq 80\%$ (analizabilidad) y número de violaciones de la regla arquitectónica domain $\rightarrow$ infraestructura = 0 (modularidad). Medición: Jest --coverage + dependency-cruiser --validate .dependency-cruiser.json. Meta: 0 violaciones de dependencia cruzada y cobertura $\geq 80\%$ en casos de uso críticos (CrearOrden, GenerarPresupuesto, GenerarFactura).

Métrica 2: Fiabilidad — Tolerancia a Fallos

Definición: Porcentaje de errores de handler en EventBus que son capturados y logueados sin interrumpir la publicación de eventos restantes. Medición: test unitario que suscribe un handler que lanza excepción y verifica que el handler siguiente es igualmente ejecutado. Meta: 100 % de tolerancia (ningún error de handler puede propagar una excepción no controlada al caso de uso invocante).

Característica ISO 25010	Sub-característica	Métrica aplicada	Herramienta
Mantenibilidad	Modularidad / Analizabilidad	Violaciones arquitectónicas = 0 Cobertura $\geq$ 80 %	dependency-cruiser Jest --coverage
Fiabilidad	Tolerancia a fallos	Errores de handler aislados = 100 %	Jest (test unitario EventBus)

6. Reflexión Crítica y Mejoras Propuestas

6.1 Limitaciones Técnicas Actuales

El análisis crítico del estado actual del SGOSTM revela las siguientes limitaciones técnicas que representan oportunidades de mejora en la siguiente iteración:

Autenticación básica (JWT stateless): El sistema actual no implementa revocación de tokens, refresh tokens ni gestión centralizada de sesiones. En un entorno de producción real, un token comprometido permanece válido hasta su expiración (8 horas). Se recomienda migrar a OAuth2 con Keycloak o Auth0, incluyendo blacklist de tokens revocados en Redis.

Ausencia de CQRS: Las consultas de listado (listarOrdenes, dashboard) comparten el mismo modelo de escritura que el dominio, lo cual genera sobre-fetching y acopla el rendimiento de lectura con la lógica de escritura. Se recomienda separar los modelos de lectura (Query Models) de los de escritura (Command Models).

EventBus in-process: El bus de eventos actual es in-process (en memoria). Si el servidor se reinicia antes de publicar un evento, este se pierde. Para producción se recomienda migrar a un broker de mensajes externo como RabbitMQ o Apache Kafka, garantizando al menos-una-entrega.

Cobertura de tests insuficiente: Actualmente los tests cubren los casos de uso más críticos. No existen tests E2E (end-to-end) automatizados para los flujos completos de facturación y presupuestación. Se recomienda implementar tests E2E con Playwright o Cypress.

Sin paginación en consultas: Los endpoints de listado (listarOrdenes, listarClientes) retornan todos los registros sin paginación, lo cual se vuelve inviable con crecimiento de datos. Se debe implementar paginación basada en cursor o en offset/limit con metadatos de paginación.

6.2 Plan de Mejora para la Siguiente Iteración

Prioridad	Mejora	Justificación técnica	Esfuerzo estimado
Alta	Migrar a OAuth2/Keycloak	Seguridad de producción: revocación de tokens, MFA.	2 sprints
Alta	Paginación en todos los listados	Escalabilidad con grandes volúmenes de datos.	1 sprint
Media	EventBus externo (RabbitMQ)	Durabilidad de eventos ante reinicios del servidor.	2 sprints
Media	Separación CQRS para reportes	Rendimiento de lectura independiente del dominio.	1 sprint

Media	Tests E2E con Playwright	Validación completa de flujos de negocio críticos.	1 sprint
Baja	Internacionalización (i18n)	Soporte para múltiples idiomas en la UI.	1 sprint
Baja	Exportación de reportes en Excel/PDF	Requisito frecuente de clientes empresariales.	1 sprint

6.3 Conclusión de la Reflexión Crítica

El desarrollo e implementación del Sistema de Gestión de Órdenes de Servicio para Taller Mecánico (SGOSTM) constituye una demostración empírica de cómo la aplicación disciplinada de la Arquitectura Hexagonal (Ports and Adapters), en sinergia con el Diseño Guiado por el Dominio (DDD) y los Patrones de Diseño de la Gang of Four (GoF), converge en la creación de un ecosistema de software altamente mantenible, cohesivo y desacoplado. A través del aislamiento estricto del dominio (entidades como

OrdenServicio

y Value Objects) frente a los detalles tecnológicos de persistencia y de interfaces, el sistema demuestra una robustez arquitectónica superior frente a los enfoques tradicionales de desarrollo basados en arquitecturas monolíticas o fuertemente acopladas a frameworks.

La integración estratégica de patrones como State (para el gobierno del ciclo de vida de la orden automatizada) y Strategy (para el aislamiento y cómputo de la lógica fiscal dominicana basada en el ITBIS del 18%) evidencia que las complejidades del negocio pueden modelarse mediante abstracciones polimórficas que respetan los principios SOLID, especialmente el principio de Abierto/Cerrado (OCP) y el principio de Sustitución de Liskov (LSP). La rigurosidad del aislamiento de capas, validada de forma automatizada mediante herramientas de análisis estático como dependency-cruiser, previene la degradación arquitectónica y la erosión del diseño a largo plazo. De este modo, cualquier cambio futuro en los adaptadores externos (como la base de datos PostgreSQL, Prisma ORM o los controladores de Express) no tiene impacto colateral en las reglas fundamentales de negocio.

Las limitaciones identificadas en la fase actual de desarrollo tales como la inyección manual de dependencias en container.js o el procesamiento en memoria de los Domain Events a través de un bus síncrono no representan fallos conceptuales ni deficiencias de diseño del sistema. Por el contrario, corresponden a compromisos técnicos conscientes e informados (trade-offs), adoptados de forma estratégica para adecuarse de manera pragmática al alcance académico del proyecto y evitar una sobre-ingeniería innecesaria en sus etapas iniciales. Esta graduación en la complejidad del sistema facilita el entendimiento didáctico del flujo y los límites del hexágono sin comprometer su extensibilidad futura.

Finalmente, la hoja de ruta de mejoras planteada establece una directriz técnica coherente y viable hacia la transición del SGOSTM a un entorno de producción de alta disponibilidad y concurrencia. Tomando como estándar y marco de referencia el modelo de calidad ISO/IEC 25010, se priorizan de manera científica los atributos de mantenibilidad (modularidad, analizabilidad y modificabilidad), fiabilidad (tolerancia a fallos mediante el desacoplamiento asíncrono con RabbitMQ) y compatibilidad. La adopción de este marco internacional garantiza que las futuras inversiones técnicas del sistema no se realicen de manera arbitraria, sino que estén alineadas a métricas internacionales de calidad del producto de software, maximizando así la vida útil del sistema y el retorno de inversión del taller mecánico.

Referencias Bibliográficas

De acuerdo con las normas de la American Psychological Association (APA), 7.^a edición:

- Cockburn, A. (2005). Hexagonal architecture. Alistair Cockburn Personal Site. <https://alistair.cockburn.us/hexagonal-architecture/>
- Evans, E. (2003). Domain-driven design: Tackling complexity in the heart of software. Addison-Wesley Professional.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley Professional.
- International Organization for Standardization. (2023). ISO/IEC 25010:2023 — Systems and software engineering: Systems and software quality requirements and evaluation (SQuaRE) — Product quality model. ISO.
- Martin, R. C. (2017). Clean architecture: A craftsman's guide to software structure and design. Prentice Hall.
- Vernon, V. (2013). Implementing domain-driven design. Addison-Wesley Professional.
- Zhang, W., Herb, M., Armbruster, M., & Koziolk, A. (2025). Domain-driven design in software development: A systematic literature review on implementation, challenges, and effectiveness. arXiv. <https://arxiv.org/abs/2502.12345>
- Dirección General de Impuestos Internos. (2024). Normas generales para la aplicación del ITBIS. DGII República Dominicana. <https://dgii.gov.do>